

Project Guide: AI-Powered Invoicing & Cash-Flow Agent

One document that walks through the whole project: what it is, how the pieces fit, and what was built each day and why. The per-day files in this folder go deeper on any single topic; this guide is the map.

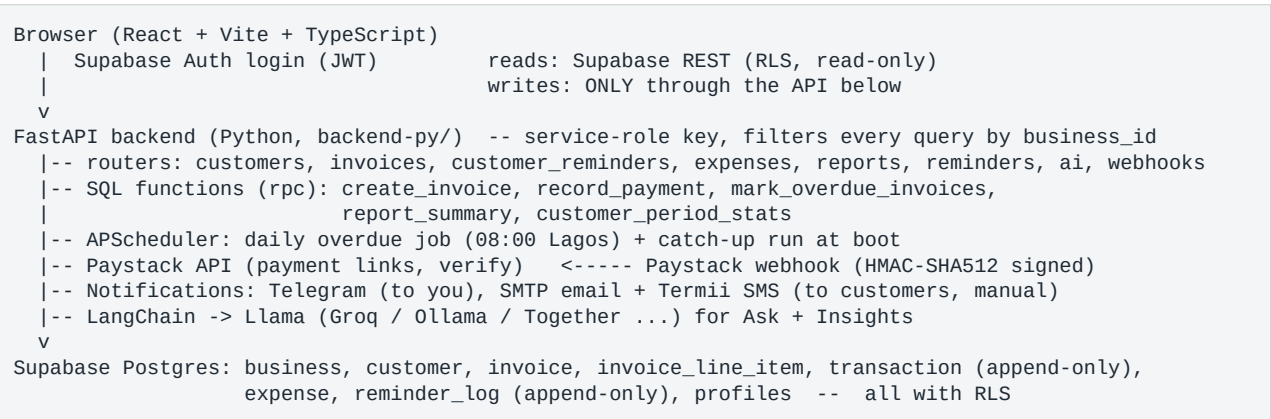
1. What this project is

A **proof-of-concept for a single Nigerian small business** that lets the owner:

- 1 Create an invoice and get a real Paystack (sandbox) payment link.
- 2 Have the invoice flip to **paid automatically** when the customer pays.
- 3 Be alerted when invoices go overdue, and remind the customer with one click (email and SMS).
- 4 Log expenses in seconds.
- 5 See one consistent set of numbers (revenue, expenses, net cash, slow payers).
- 6 **Ask questions in plain English** and get an **AI-written insight summary**, both grounded strictly in the business's own computed numbers.

Deliberately out of scope: multiple businesses per owner, user roles, bank-statement import, open banking, WhatsApp, customer-facing chat.

2. The architecture at a glance



Technology choices in one table

Layer	Choice	Why
Backend	Python + FastAPI	You code in Python; Decimal for money, pydantic validation, zoneinfo timezones
Frontend	React + Vite + TypeScript	Browsers only run JavaScript; TypeScript catches money-handling mistakes
Database	Supabase Postgres	Auth + database + RLS in one; migrations give a clean history
Money logic	SQL functions	One transaction, next to the data, safe under concurrency
Payments	Paystack sandbox	Real payment rails, test money

Layer	Choice	Why
AI	Llama via LangChain <code>ChatOpenAI(base_url=...)</code>	Open model, host-agnostic: change 3 env lines to switch Groq / Ollama / Together

3. The rules that shape every decision

- 1 Webhook signatures are always verified, and payments are recorded idempotently.** Webhooks are delivered more than once; the database, not application code, prevents double-counting.
- 2 Money is computed once, in SQL.** The dashboard and the AI read the same numbers, so they cannot disagree.
- 3 The model only reasons over numbers we hand it.** It never queries the database and never calculates the business's figures. If the answer is not in the data it must say so.
- 4 Financial records are immutable.** Transactions and reminder logs are append-only (enforced by database triggers). Corrections happen through a `void` status or new records.
- 5 Prices are snapshots.** A line item's `unit_price` is written once and never recalculated.
- 6 Browsers can read (through RLS) but not write.** Every write goes through the API, which computes totals, numbers and links itself.

4. Day by day

Day 1: Foundation (schema, auth, skeleton)

Goal: a backend that authenticates and reads/writes a real, RLS-protected schema.

- Tables: `business`, `customer`, `invoice`, `invoice_line_item`, `transaction`, `expense`, plus `profiles` linked to `auth.users` by trigger.
- RLS on every table, checking that the row's business belongs to `auth.uid()`.
- The backend uses the **service-role key**, which bypasses RLS by design for trusted server work; that is why every backend query must filter by `business_id`.
- React frontend with login and a dashboard shell.
- *Later fix (Day 9):* `profiles` had no RLS and browsers could insert invoices directly; both were closed.

Day 2: Invoice creation and payment link (`README_day2.md`)

Goal: create an invoice in the UI and receive a clickable sandbox link.

- Flow: form -> `POST /customers` (find or create by email) -> `POST /invoices` -> validate -> SQL `create_invoice` (one transaction: number, invoice, line items, total) -> Paystack initialise -> save reference and checkout URL -> status `draft` to `sent`.
- **Totals computed in SQL**, never trusted from the client.
- **Kobo/naira** handled once, in `money.py`, with `Decimal` (floats give $19.99 * 100 = 1998.99999\dots$).
- **Due date = end of that day in Lagos**, so invoices don't go overdue a day early.
- If Paystack is down the invoice is saved as `draft` and the link can be retried; an existing link is never replaced.
- The Day 1/2 code had import errors, no auth on `/invoices`, and never created a `business` row; all fixed.

Day 3: Webhook and automatic confirmation (`README_day3.md`)

Goal: paying the link flips the invoice to **paid** with no manual step.

- **POST /webhooks/paystack** reads the **raw body bytes**, verifies HMAC-SHA512 (Paystack signs with the secret key; there is no separate webhook secret) using **hmac.compare_digest**.
- Only **charge.success** in NGN is acted on. SQL **record_payment** locks the invoice row and inserts the transaction with **UNIQUE(provider_reference)** and **ON CONFLICT DO NOTHING**, so duplicates cannot double-count.
- Status codes are deliberate: **401** forged signature, **200** duplicate/unknown/ignored (avoids retry storms), **500** database failure (the only case where Paystack should retry).
- "Check payment" verifies through Paystack's API and uses the same SQL function, a safe stand-in when localhost cannot receive webhooks.

Day 4: Overdue alerts (**README_day4.md**)

Goal: an unpaid invoice past its due date raises an alert without you looking.

- SQL **mark_overdue_invoices()** flips **sent** to **overdue** in one statement.
- APScheduler runs daily at 08:00 Lagos time, plus a **catch-up run at boot** for hosts that sleep; **misfire_grace_time** and **coalesce** make it forgiving.
- Alerts go to **you**, as one digest: the dashboard banner (computed from **due_date**, so always right), Telegram if configured, otherwise the log.
- **reminder_sent_at** is stamped only after a successful send, so a Telegram outage retries next run.
- A job failure is logged and shown in **/jobs/status**; it never crashes the API.

Day 5: Expense logging (**README_day5.md**)

Goal: record money out in under 10 seconds.

- Fixed category enum enforced in the UI, pydantic and Postgres; the AI never guesses categories.
- Category and date persist between entries; the amount field is auto-focused.
- A picked date is stored as noon Lagos time so it stays inside that calendar day.
- Running total computed with **Decimal** on the server. Expenses are immutable.

Day 6: Aggregation layer (**README_day6.md**)

Goal: one endpoint returns every number the dashboard and AI use: **GET /reports/summary?range=week|month**.

- All arithmetic in **one SQL function**, **report_summary**.
- Timezone is **Africa/Lagos**, set explicitly; the database only ever sees absolute UTC instants, with half-open **[from, to)** windows.
- **Revenue is cash-basis** (money received in the window), separate from **outstanding_now**.
- **include_previous=true** compares like-for-like (same elapsed time into the previous period), which feeds Day 8.

Customer reminders (added feature) (**README_customer_reminders.md**)

Goal: a "Remind customer" button beside each unpaid invoice.

- Manual only, after a confirm dialog naming the recipient; the message includes the amount still owed and the Paystack link.
- Email through plain SMTP; SMS through Termii; phone numbers normalised to **234...**
- 24-hour cooldown per invoice; every attempt is written to the append-only **reminder_log**.

- Channels fail independently and the response says what each did.

Day 7: Llama-powered questions ([README_day7.md](#))

Goal: ask a money question in English, get a grounded answer, and a refusal when the data lacks it.

- `POST /ask` builds fresh aggregates, then runs a LangChain chain: `ChatPromptTemplate | ChatOpenAI(base_url=...) | StrOutputParser`.
- The prompt says: use only supplied numbers, say "I don't have that data" otherwise, never estimate; the question is wrapped in tags to blunt prompt injection; `temperature=0`.
- After answering, code flags any money-sized number that appears nowhere in the data (`unverified_figures`), which matters more with open models.
- Rate limit of 20 AI requests per user per 10 minutes; provider failures become friendly errors.
- **The key test:** a trick question (for example "predict next month's revenue") must be declined.

Day 8: Narrative insights ([README_day8.md](#))

Goal: one click gives 2 to 3 observations beyond what the dashboard shows.

- `POST /insights` sends current and previous periods, **percentage changes computed by code**, and per-customer stats for both periods (SQL `customer_period_stats`), so it can notice "your best customer now pays slower".
- The prompt bans restating numbers and allows fewer than three observations rather than filler.
- Bullet lines are parsed and capped at 3; results are cached 15 minutes keyed by a fingerprint of the data.

Day 9: Hardening and polish ([README_day9.md](#))

Goal: survive misuse and feel like one product.

- Misuse table: forged signatures (401), malformed JSON (400), invalid invoices (400), missing tokens (401), job failures (logged), provider outages (friendly errors), AI spam (429).
- `supabase/tests/rls_isolation.sql` creates two businesses and proves one cannot see or write the other's data. It exposed the two Day 1 holes, fixed by the `day9_rls_hardening` migration: browsers are now read-only.
- One dashboard: banner, summary, Ask and Insights, then invoices and expenses.

Day 10: Buffer and demo ([README_day10.md](#), [DEMO_SCRIPT.md](#))

Goal: run the demo without thinking about it.

- `seed_demo.sql` loads 5 customers, 16 invoices and 13 expenses with a built-in story (Ada slowing from about 2 to 7 days; overdue money at Chidi and Emeka); `cleanup_demo.sql` removes it.
- Demo path (about 5 minutes): create an invoice, pay it in the sandbox, watch it flip to paid, ask a question, generate insights.
- A ready-made paragraph explains what is real (payment rails, webhook, database, AI reasoning) versus simulated (test-mode money, demo data).

5. Where things live

Path	Contents
backend-py/app/	main.py, config.py, auth.py, ai.py, analytics.py, notifier.py, paystack.py, period.py, money.py, dates.py, schemas.py, routers/, jobs/
backend-py/tests/	66 tests with external services mocked
frontend/src/	App.tsx, api/, components/ (InvoiceForm, InvoiceList, OverdueBanner, ExpenseSection, SummaryPanel, AskPanel, InsightsPanel), lib/
supabase/migrations/	schema, hardening, create_invoice, record_payment, overdue, report_summary, reminders, customer_period_stats, RLS hardening
supabase/	seed_demo.sql, cleanup_demo.sql, tests/rls_isolation.sql
docs/	this guide, the per-day READMEs, DEMO_SCRIPT.md

6. Running it

See the top-level `README.md` for the setup checklist, run commands and environment variables. In short: `supabase db push`, fill in `backend-py/.env` and `frontend/.env`, `pip install -r requirements.txt`, run `uvicorn app.main:app --reload --port 4000`, then `npm run dev` in `frontend/`.

7. What is verified and what is not

- **Verified:** all 66 tests pass; the frontend type-checks and builds; every SQL function, the seed and cleanup scripts and the RLS test were run on a scratch Postgres; the Llama client was tested against a local OpenAI-compatible stand-in server.
- **Not verified live:** Supabase, Paystack, Groq, SMTP, Termii and Telegram were exercised only through mocks. Your first real run of each is the true test, especially the AI trick question.
- **Not built:** deployment files, WhatsApp (needs a paid business account and approved templates), email over implicit SSL (port 465), multi-process rate limiting (the limiter and insights cache are in memory).

8. Glossary

- **RLS (Row Level Security):** Postgres rules that decide which rows a logged-in user can see or change.
- **Service-role key:** a Supabase key that bypasses RLS; it must stay on the server.
- **Idempotent:** doing the same operation twice has the same effect as doing it once.
- **Cash basis:** revenue counted when money is received, not when the invoice is issued.
- **LCEL:** LangChain's `prompt | model | parser` composition syntax.
- **Kobo:** one hundredth of a naira; Paystack amounts are in kobo.